



DOCKER FOR AI PROJECTS

Ship Your RAG App, Not Just Run It

One real service, containerized step by step
from FROM to docker run



AI Coach John



PROITBRIDGE
Strive For Better Future

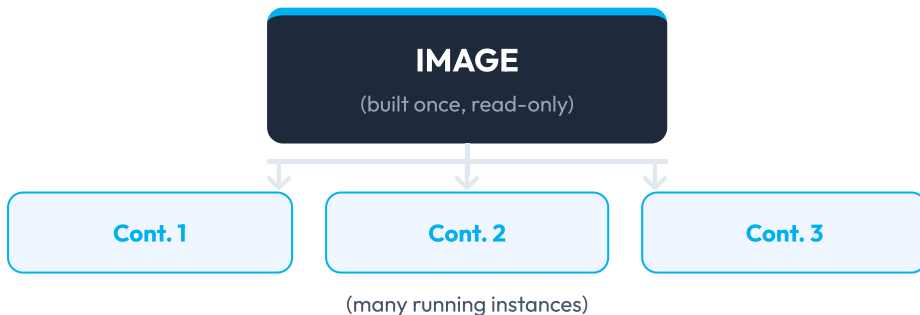
Follow

↻ Repost

The one distinction that confuses everyone at the start

Docker packages your app *and* everything it needs to run—Python version, system libraries, dependencies, config—into a single portable unit.

- **Image:** the blueprint. A read-only, frozen snapshot of your app + env. Built once, never changes.
- **Container:** a running instance. You can start many containers from one image.



⚠ WHY DOCKER?

AI dependencies are the messiest in software

- × **Heavy, version-sensitive:** torch, transformers, CUDA. A mismatch silently breaks things instead of erroring cleanly.
- × **"Works on my machine" is worse:** Apple Silicon wheels vs Linux server wheels.
- × **Must ship together:** RAG needs its vector index, embedding model, and code all consistent.
- × **Reproducibility:** Can't trust eval comparisons if the env silently changed.

WITHOUT DOCKER

- × `"pip install failed"`
- × `"wrong CUDA version"`
- × `"works on Mac only"`
- × `"which Python was it?"`

WITH DOCKER

- ✓ `docker run`
- ✓ `docker run`
- ✓ `docker run`
- ✓ `docker run`



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

The service we'll containerize across this carousel

 rag-service/
  app/
  main.py
  rag_pipeline.py
  config.py
  data/
  faiss_index/
  requirements.txt
  .env (secrets)
  .dockerignore
  Dockerfile

```
# app/main.py

from fastapi import FastAPI
from app.rag_pipeline import answer

app = FastAPI()

@app.post("/query")
def query(question: str):
    return {"answer":
        answer(question)}
```



The instructions you'll **actually** use

FROM	The base image you build on top of
WORKDIR	Sets the working directory inside
COPY	Copies files from host to image
RUN	Executes command at build time
ENV / EXPOSE	Sets variables / documents port
CMD	Default command at run time

BUILD TIME (docker build)

FROM, WORKDIR, COPY,
RUN, ENV, EXPOSE



image created

RUN TIME (docker run)

CMD / ENTRYPOINT



container starts



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 Repost

Choosing the Right Base Image: Your first line matters

```
# Too big – ~1GB before you install anything
FROM python:3.11

# Good default for AI projects – ~150MB, Debian-based
FROM python:3.11-slim

# Tempting but painful – breaks/slows ML wheels
FROM python:3.11-alpine # avoid for AI projects

# If you need GPU/CUDA
FROM nvidia/cuda:12.1.0-runtime-ubuntu22.04
```

python:3.11		~1GB	✗	bloated
python:3.11-slim		~150MB	✓	use this
python:3.11-alpine		~50MB	✗	breaks ML wheels
nvidia/cuda:...		~2GB	✓	if GPU needed



AI Coach John



Follow

↻ Repost

A First Working Dockerfile: Naive but functional

```
FROM python:3.11-slim
WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

THE INEFFICIENCY

Edit one line of `main.py`



`COPY . .` → cache INVALIDATED



`RUN pip install` → reinstalls EVERYTHING (5+ mins) 🕒



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↺ Repost

🔗 STEP 3

Layer Caching: The Big Speed Win

Copy dependencies **before** code. Docker reuses cached layers until one changes. By copying `requirements.txt` first, code edits won't trigger pip reinstalls.

```
# 1. Copy ONLY the dependency file first
COPY requirements.txt .

# 2. Install deps – cached until requirements change
RUN pip install --no-cache-dir -r requirements.txt

# 3. NOW copy your application code
COPY app/ ./app/
```

THE SPEED FIX

Edit `main.py`



COPY req.txt

✓ CACHED (unchanged)

RUN pip install

✓ CACHED (reused, 0s)

COPY app/

⚡ rebuild only this layer

(5 minutes → 3 seconds)



AI Coach John



PROFITBRIDGE
Strive For Better Future

Follow

🔄 Repost

🔒 STEP 4

.dockerignore: Keep junk & secrets out

```
# .dockerignore
.venv/ __pycache__/ .env # NEVER
bake secrets .git/ data/raw/ #
huge unneeded files
```

Without this, `COPY . .` drags your virtualenv, git history, and **.env file** into the image.

Anyone who pulls that image can extract your API keys.

WITHOUT .dockerignore

app/	✓
.venv/ (400MB)	✗
.git/ (200MB)	✗
.env	⚠ SECRETS

image: 1.2GB ⚠

WITH .dockerignore

app/	✓
data/faiss/	✓

image: 180MB ✓



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

🔄 Repost

🔒 STEP 5

Handling Secrets Correctly: Runtime, never build time

```
# WRONG — key is permanently baked into an image layer
ENV OPENAI_API_KEY=sk-abc123...

# RIGHT — declare variable, supply value at runtime
ENV OPENAI_API_KEY=""
```

Docker images are made of inspectable layers. A secret written with ENV at build time stays in the image history forever. Always inject secrets at docker run time.

BUILD TIME (Dockerfile)

🔒 no secrets baked in
(image is safe to share)



RUN TIME (CLI)

-e API_KEY=... 🔒
(secret stays local)



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

Multi-Stage Builds: Shrinking the Image

Compilers (gcc, g++) are needed to build some ML packages but are useless at runtime. A multi-stage build compiles in stage 1, then copies only the resulting packages into a clean stage 2.

```
# ----- Stage 1: builder -----
FROM python:3.11-slim AS builder
RUN apt-get update && apt-get install -y gcc g++
COPY requirements.txt .
RUN pip install --prefix=/install -r requirements.txt

# ----- Stage 2: runtime -----
FROM python:3.11-slim
# copy ONLY installed packages, leave gcc behind
COPY --from=builder /install /usr/local
COPY app/ ./app/
```

STAGE 1(builder)

gcc, g++, headers
pip build deps
python packages

~900MB (discarded)

packages
only →

STAGE 2 (final)

python packages ✓
app code ✓

~350MB (ships)



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

🔄 Repost

Running as a Non-Root User:

The fix people skip

By default, containers run as root. If your AI agent (which executes generated code) is compromised, running as root makes an escape far more damaging.

```
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ ./app/

# Create non-root user and hand over ownership
RUN useradd --create-home appuser && chown -R appuser:appuser /app
USER appuser

CMD ["uvicorn", ...]
```

RUN pip install

 needs root

USER appuser

 switch AFTER installs

CMD uvicorn ...

 runs unprivileged



AI Coach John



Follow

 Repost

💚 STEP 8

Health Checks: Don't let Docker kill slow startups

Loading an embedding model or FAISS index into memory can take 30-60s at startup. Without a generous start-period, your orchestrator kills the container as "unhealthy" before it finishes loading.

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=60s \  
CMD curl -f http://localhost:8000/health || exit 1
```

THE STARTUP TIMELINE

0-60s: loading model + index	[no checks yet]
60s+: /health returns 200 ✓	[checks every 30s]

⚠ Without start-period: killed at 30s, restart loop forever



AI Coach John



PROFITBRIDGE
Strive For Better Future

Follow

↻ Repost

Persisting Data with Volumes: Containers are ephemeral

Anything written *inside* a container disappears when it's removed. For AI, you must mount volumes for your vector index and downloaded model weights (so you don't re-download GBs on restart).

```
# Mount a host directory into the container
docker run -p 8000:8000 \
  -v $(pwd)/data/faiss_index:/app/data/faiss_index \
  rag-service
```

HOST MACHINE

./data/faiss_index
(persists forever)

CONTAINER

/app/data/faiss_index
(deleted on exit)

 mounted together



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 Repost

Docker Compose: Real RAG is rarely one container

```
services:
  rag-api:
    build: .
    depends_on: [qdrant, ollama]

  qdrant: # vector database
    image: qdrant/qdrant:latest
    volumes:
      - qdrant_storage:/qdrant/storage

  ollama: # local LLM server
    image: ollama/ollama:latest
    volumes:
      - ollama_models:/root/.ollama
```

docker compose up



rag-api
:8000

qdrant
:6333

ollama
:11434

Shared network. App connects to **http://ollama:11434**, not localhost.



AI Coach John



Follow

 Repost

GPU Support: When your container needs the GPU

Use a runtime CUDA base image (not devel). Match your CUDA version to what your PyTorch build expects, or it will silently fall back to CPU.

```
FROM nvidia/cuda:12.1.0-runtime-ubuntu22.04

# ... install python and pip ...

CMD ["uvicorn", "app.main:app"]
```

RUNNING WITH GPU_s

NVIDIA GPU
Container Toolkit

--gpus all →

CUDA runtime
torch (GPU build)



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 Repost

Everything from this carousel, assembled

```
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.11-slim
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1
WORKDIR /app
COPY --from=builder /install /usr/local
COPY app/ ./app/
COPY data/ ./data/

RUN useradd --create-home appuser && chown -R appuser:appuser /app
USER appuser

EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --start-period=60s \
  CMD curl -f http://localhost:8000/health || exit 1

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

(PYTHONUNBUFFERED=1 ensures Python logs stream directly to docker logs)



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost



The commands you'll **run** constantly

Build & Run

```
docker build -t rag-service .  
docker run -p 8000:8000 --env-file .env rag-service  
docker run -d -p 8000:8000 rag-service # detached
```

Inspect

```
docker ps # running containers  
docker images # local images  
docker logs -f <id> # follow logs live  
docker exec -it <id> bash # shell into container
```

Clean Up (AI images get big fast)

```
docker rm <id> # remove container  
docker rmi <image_id> # remove image  
docker system prune -a # remove all unused
```

Compose

```
docker compose up --build  
docker compose down -v # stops & removes volumes
```



Mistakes that cost hours:

Common Pitfalls

⚠ Code before deps

Busts the dependency cache; every rebuild reinstalls torch (Slide 8).

⚠ Alpine base image

musl libc breaks precompiled ML wheels; builds fail or take forever.

⚠ Secrets in ENV

Permanently visible in image layers, even if later removed.

⚠ No .dockerignore

Ships your .venv, .git, and .env into the image.

⚠ No --start-period

Model load takes 60s, healthcheck kills it at 30s → infinite loop.

⚠ Running as root

Dangerous for AI agents that execute generated code.

⚠ Missing 0.0.0.0

Binding to 127.0.0.1 inside a container makes it unreachable.

⚠ Weights in image

A 40GB image is unusable. Mount them as a volume instead.

